

PLANTILLA DE DESARROLLO DE
TRABAJO PRÁCTICO UNIDAD
DIDÁCTICA 4
U-TAD ONLINE

ÍNDICE

1. ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA	3
2. DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA	3
3. ARCHIVOS ADJUNTOS DE LA ACTIVIDAD	3
4. RUBRICA DE CORRECCIÓN	3
5. ANEXO I	4
NOMENCLATURA DE ARCHIVOS:	
.....	4

1. ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA

Nombre de la tarea	Tarea Múltiple de la Unidad 4
Tipo de tarea	Individual
Entregables	Un fichero pdf con el ejercicio a mano y pantallazos de los otros dos circuitos, Ficheros .circ de los circuitos

2. DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA

La actividad Práctica de la unidad 3 consta de tres partes claramente diferenciadas. En el documento pdf donde se suba el ejercicios 1 hecho a mano también se incluirán unos pantallazos de los circuitos de los ejercicios 2 y 3. Recordad que en caso de emplear algún tipo de ayuda con IA, es necesario declararla en la memoria, indicando modelo y versión de IA, prompt, resultado del prompt y vuestro entendimiento acerca de esa respuesta dada por la IA.

1 Ejercicio realizado a mano

Este ejercicio debe realizarse a mano sin usar el simulador logisim:

Diseñe un circuito lógico que convierta números de 3 bits representados en signo-magnitud (S-M) a su equivalente en complemento a 1.

Asuma que se emplea el bit más significativo de los 3 bits del S-M para el signo. Además, cada bit debe ser nombrado de forma específica:

- SMSigno: el bit del signo de la representación SM
- SM1: el bit siguiente al del signo en SM
- SM0: el bit menos significativo de la representación SM
- C2: el bit más significativo de los 3 bits de la representación complemento a 1
- C1: el bit siguiente al C2 de la representación en complemento a 1 • C0: el bit menos significativo de la representación en complemento a 1
- Se pide:

Se pide:

- La tabla de verdad
- Las funciones algebraicas sin simplificar
- Las funciones algebraicas simplificadas por Karnaugh
- El circuito simplificado del bit C2, C1 y C0

2 Ejercicio de Logisim 1 Combinacional

Diseño de multiplexor 8x1 usando multiplexores de 4x1 y de 2x1

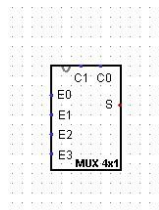
El multiplexor tiene 8 entradas (D0–D7) y una salida (Y), usando datos de 1 bit.

Deberéis usar:

- Dos multiplexores 4x1 **diseñados por vosotros con puertas lógicas básicas** (NO se puede usar el multiplexor de Logisim).
- Un multiplexor 2x1 (**aquí SÍ se puede usar el componente estándar de Logisim**).

Pasos a realizar:

- A)** Crear un circuito con puertas lógicas que funcione como multiplexor 4x1 (E0–E3 como entradas, S como salida, y C0, C1 como selectores). Además, presentarlo con aspecto de chip, similar al ejemplo mostrado.



- B)** En el mismo archivo .circ, crear un circuito principal con:

- Dos MUX 4x1 hechos por vosotros,
- Un MUX 2x1 (Logisim),
- 8 entradas de datos (D0–D7), • 3 selectores (C0, C1, C2),

Todos conectados de forma que funcionen como un multiplexor 8x1 completo.

3 Ejercicio de Logisim 2

Vamos a realizar un circuito digital con la herramienta logisim, que emule el temporizador de una máquina de café. Se puede hacer todo en un solo circuito (no hace falta emplear subcircuitos)

Descripción:

Nuestra máquina acepta cápsulas para preparar 4 tipos de cafés (cada tipo tiene un código binario). Según el tipo de café elegido la máquina debe estar echando agua sobre la cápsula un tiempo diferente que mostramos a continuación:

00 -> Café solo -> 3 segundos de agua

01 -> Café cortado -> 4 segundos de agua

10 -> Café con Leche -> 5 segundos de agua

11 -> Café americano -> 7 segundos de agua

Queremos implementar el circuito digital que va a controlar este temporizador.

Para ello debemos realizar los siguientes pasos:

1 Necesitamos dos bits de entrada, cuya combinación nos da el tipo de café. Esos dos bits son la entrada de circuito (se puede usar alguno de los ya disponibles en logisim). Dependiendo de su valor, el circuito nos devuelve un código binario de 3 bits que representa los segundos que la máquina va a estar echado agua. Es decir, esta primera parte del circuito si recibe un 01, que equivale a un café cortado, devolverá 4 en binario con 3 bits que es 100.

2 Una vez esté montado el paso anterior, toca crear un circuito contador (hay que realizarlo con biestables, no se puede usar en este caso el contador de logisim). El circuito contador se activa cuando se aprieta un botón/pulsador. Al apretarse las acciones que se realizan son:

2.1 Se resetean los biestables que conforman el contador a cero

2.2 Comienza el contador. El número de la cuenta que está representado en los bits de los biestables, hay que representarlo mediante un "Hex digit display" proporcionado por logisim. Es parecido al display de 7 segmentos solo que recibe un número de 4 bits y lo representa en formato decimal con leds. Tened en cuenta que este display recibe 4 bits y nosotros tenemos en el contador 3. Por tanto habrá que pasar al display un 0 "hardcodeado" como cifra más significativa. Por ejemplo: si queremos representar el 3 que en binario es 011, al display le mandaremos un 0011.

2.3 El valor que está obteniendo el contador de forma secuencial, se tiene que comparar con el límite que nos da el circuito del paso 1. Es decir, el contador se va incrementando en 1 y cuando alcance el valor que nos ha devuelto el circuito del paso 1, el contador se para y se queda en ese número.

1 Ejercicio realizado a mano

Este ejercicio debe realizarse a mano sin usar el simulador logisim:

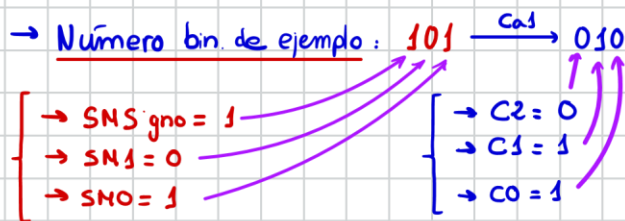
Diseñe un circuito lógico que convierta números de 3 bits representados en signo-magnitud (S-M) a su equivalente en complemento a 1.

Asuma que se emplea el bit más significativo de los 3 bits del S-M para el signo. Además, cada bit debe ser nombrado de forma específica:

- SMSigno: el bit del signo de la representación SM
- SM1: el bit siguiente al del signo en SM
- SM0: el bit menos significativo de la representación SM
- C2: el bit más significativo de los 3 bits de la representación complemento a 1
- C1: el bit siguiente al C2 de la representación en complemento a 1
- C0: el bit menos significativo de la representación en complemento a 1
- Se pide:

Se pide:

- La tabla de verdad
- Las funciones algebraicas sin simplificar
- Las funciones algebraicas simplificadas por Karnaugh
- El circuito simplificado del bit C2, C1 y C0



Al ser complemento a 1, si el número es positivo, se representa igual, pero si es negativo, se invierten todos los bits

a) Tabla de verdad

SMSig	SM1	SM2	C2	C1	C0	Núm.
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	0	0	1	0	2
0	1	1	0	1	1	3
1	0	0	0	1	1	-0
1	0	1	0	1	0	-1
1	1	0	0	0	1	-2
1	1	1	0	0	0	-3

Al ser positivo (SMSigno = 0), en complemento a 1, dejamos el número como está.

Funciones algebraicas simplificadas

→ Miramos salida C2: Nos damos cuenta de que C2 no cambia aunque la entrada cambie (siempre es 0)

SE ELIMINA, YA QUE NUNCA CAMBIA

→ Miramos salida C1:

SNSigno	SM ₁	C ₁
0	0	0
0	0	0
0	1	1
0	1	1
1	0	1
1	0	1
1	1	0
1	1	0

Escribimos un término por cada fila que C₁ es igual a 1 (SOP)

0	0	0
0	1	1
1	0	1
1	1	0

Claramente

XOR

$$(TSNSigno \cdot SM_1) + (SNSigno \cdot TSM_1)$$

→ Miramos salida C0:

SNSigno	SM ₀	C ₀
0	0	0
0	1	1
0	0	0
0	1	1
1	0	1
1	1	0
1	0	1
1	1	0

→ Dos filas sólo son repetidas, si el valor tanto de entradas como salidas es igual

SNSigno	SM ₀	C ₀
0	0	0
0	1	1
1	0	1
1	1	0

Claramente
XOR

$$C_0 = (TSNSigno \cdot SM_0) + (SNSigno \cdot TSM_0)$$

Mapas de Karnaugh

→ Los mapas de Karnaugh, se utilizan para justificar gráficamente la expresión algebraica

• MAPA PARA C1

SNSigno	SM ₁	C ₁
0	0	0
0	1	1
1	0	1
1	1	0

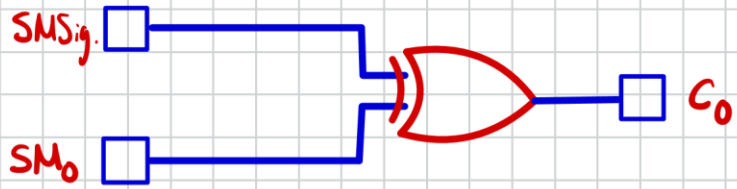
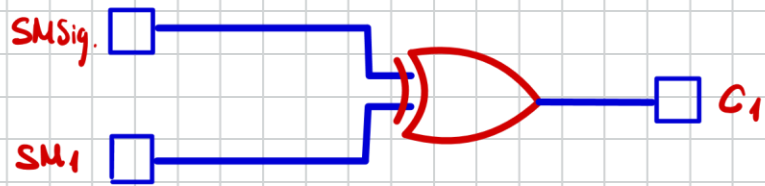
SNSigno	SM ₁
	0 1
0	0 1
1	1 0

• MAPA PARA C0

SNSigno	SM ₀	C ₀
0	0	0
0	1	1
1	0	1
1	1	0

SNSigno	SM ₀
	0 1
0	0 1
1	1 0

Circuito simplificado del bit C2 C1 y C0



C_2 ☐ ← Siempre es 0

2 Ejercicio de Logisim 1 Combinacional

Diseño de multiplexor 8x1 usando multiplexores de 4x1 y de 2x1

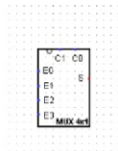
El multiplexor tiene 8 entradas (D0–D7) y una salida (Y), usando datos de 1 bit.

Deberéis usar:

- Dos multiplexores 4x1 **diseñados por vosotros con puertas lógicas básicas** (NO se puede usar el multiplexor de Logisim).
- Un multiplexor 2x1 (**aquí SÍ se puede usar el componente estándar de Logisim**).

Pasos a realizar:

- A) Crear un circuito con puertas lógicas que funcione como multiplexor 4x1 (E0–E3 como entradas, S como salida, y C0, C1 como selectores). Además, presentarlo con aspecto de chip, similar al ejemplo mostrado.



- B) En el mismo archivo .circ, crear un circuito principal con:

- Dos MUX 4x1 hechos por vosotros,
- Un MUX 2x1 (Logisim),
- 8 entradas de datos (D0–D7),
- 3 selectores (C0, C1, C2),

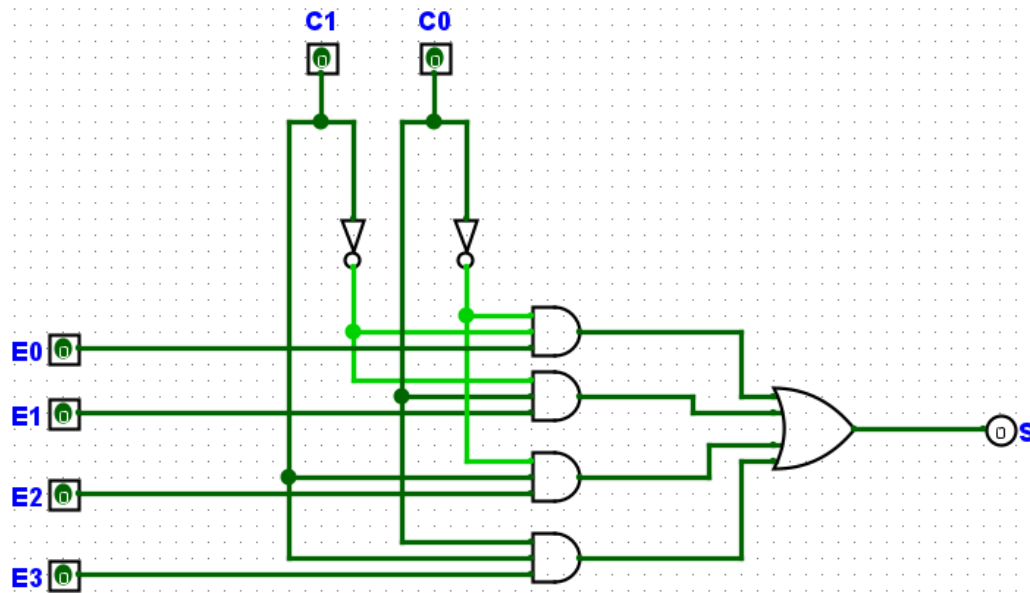
Todos conectados de forma que funcionen como un multiplexor 8x1 completo.

Tablas de multiplexores

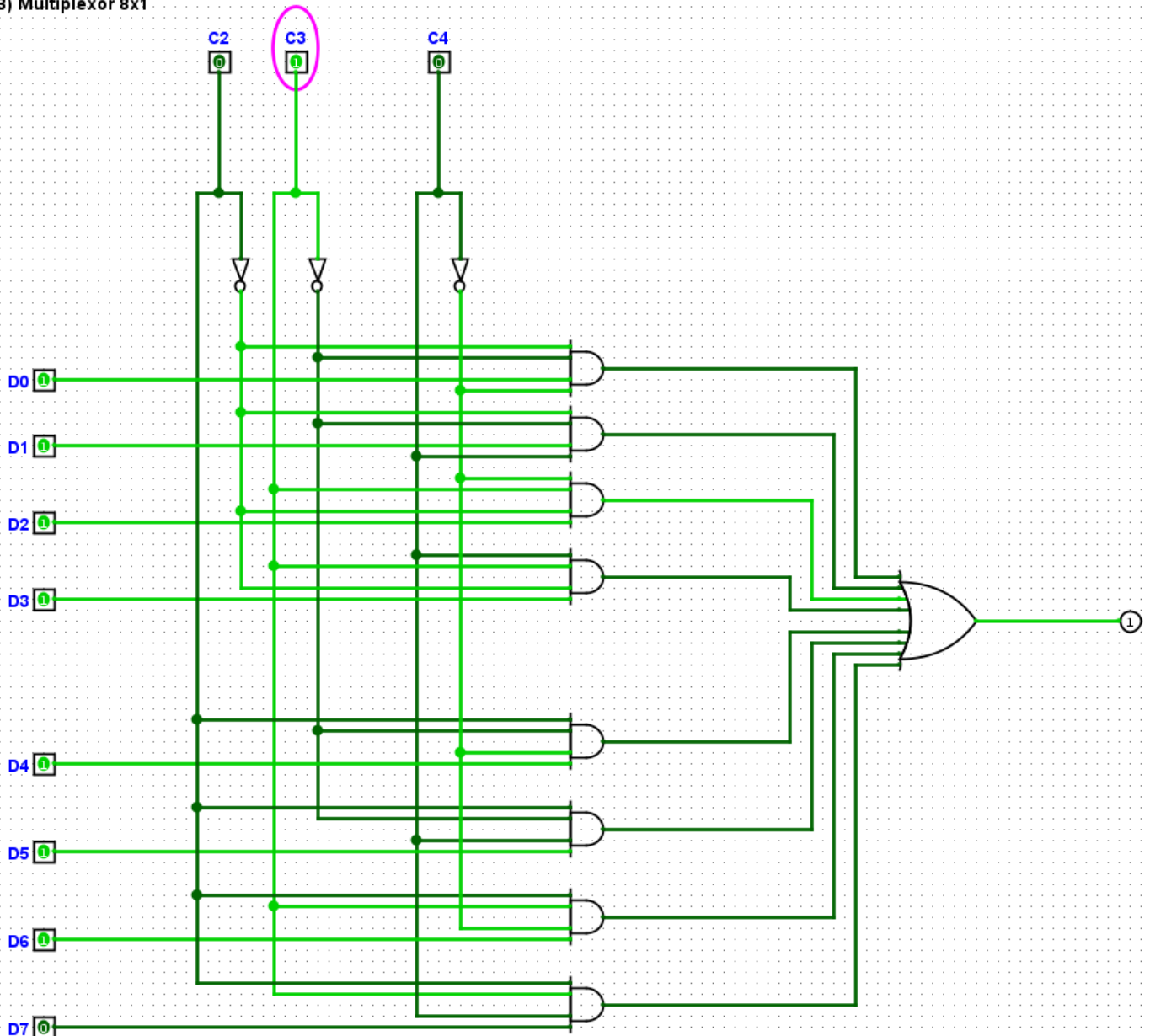
C1	C0	S
0	0	E0
0	1	E1
1	0	E2
1	1	E3

C2	C3	C4	S
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

A) Circuito que simule un multiplexor 4x1 (E0-E3 como entradas, S como salida y C0 y C1 como selectores)



B) Multiplexor 8x1



NOTA: Para el apartado B, he intentado meter un MUX 2x1, cómo dice el ejercicio, pero no he logrado hacer que la lógica funcione correctamente.

3 Ejercicio de Logisim 2

Vamos a realizar un circuito digital con la herramienta logisim, que emule el temporizador de una máquina de café. Se puede hacer todo en un solo circuito (no hace falta emplear subcircuitos)

Descripción:

Nuestra máquina acepta cápsulas para preparar 4 tipos de cafés (cada tipo tiene un código binario). Según el tipo de café elegido la máquina debe estar echando agua sobre la cápsula un tiempo diferente que mostramos a continuación:

00 -> Café solo -> 3 segundos de agua → $011 = 3s$

01 -> Café cortado -> 4 segundos de agua → $100 = 4s$

10 -> Café con Leche -> 5 segundos de agua → $101 = 5s$

11 -> Café americano -> 7 segundos de agua → $111 = 7s$

Queremos implementar el circuito digital que va a controlar este temporizador.

Para ello debemos realizar los siguientes pasos:

1 Necesitamos dos bits de entrada, cuya combinación nos da el tipo de café. Esos dos bits son la entrada de circuito (se puede usar alguno de los ya disponibles en logisim). Dependiendo de su valor, el circuito nos devuelve un código binario de 3 bits que representa los segundos que la máquina va a estar echando agua. Es decir, esta primera parte del circuito si recibe un 01, que equivale a un café cortado, devolverá 4 en binario con 3 bits que es 100.

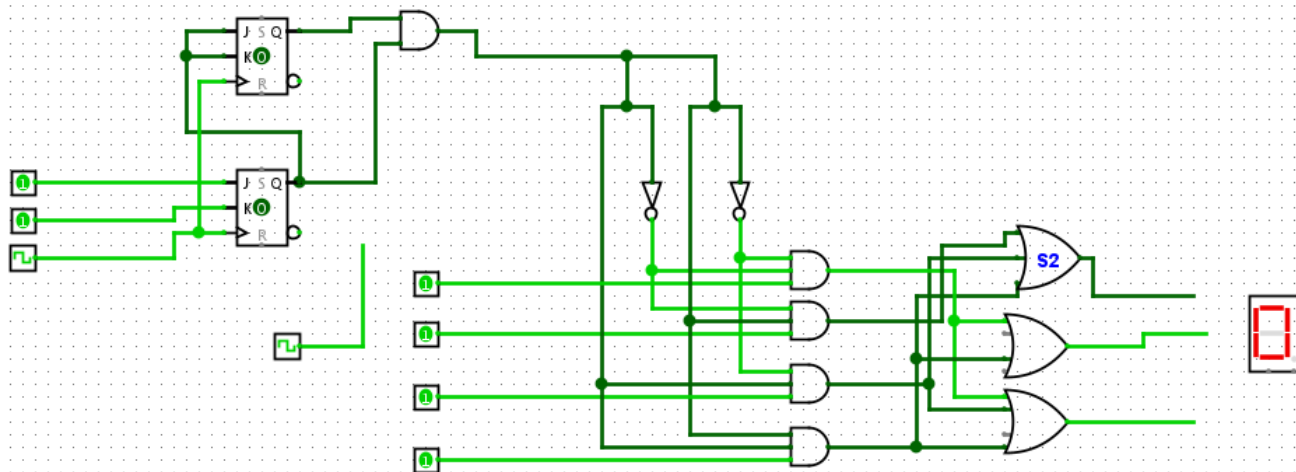
2 Una vez esté montado el paso anterior, toca crear un circuito contador (hay que realizarlo con biestables, no se puede usar en este caso el contador de logisim). El circuito contador se activa cuando se aprieta un botón/pulsador. Al apretarse las acciones que se realizan son:

2.1 Se resetean los biestables que conforman el contador a cero

2.2 Comienza el contador. El número de la cuenta que está representado en los bits de los biestables, hay que representarlo mediante un "Hex digit display" proporcionado por logisim. Es parecido al display de 7 segmentos solo que recibe un número de 4 bits y lo representa en formato decimal con leds. Tened en cuenta que este display recibe 4 bits y nosotros tenemos en el contador 3. Por tanto habrá que pasar al display un 0 "hardcodeado" como cifra más significativa. Por ejemplo: si queremos representar el 3 que en binario es 011, al display le mandaremos un 0011.

2.3 El valor que está obteniendo el contador de forma secuencial, se tiene que comparar con el límite que nos da el circuito del paso 1. Es decir, el contador se va incrementando en 1 y cuando alcance el valor que nos ha devuelto el circuito del paso 1, el contador se para y se queda en ese número.

3) Ejercicio máquina de café



NOTA: Esto es lo máximo que he podido conseguir, me he liado con la lógica. Para la realización de este ejercicio, se ha utilizado la ayuda de la IA Copilot.